



OPTATIVA Machine Learning

Autor/a: Núria Pujol Vilanova | Curs: 2025-2026

Institut TIC Barcelona

Cadenes de caràcters (*strings*), entrada de dades i sortida per pantalla

Declaració de Strings en Python

En Python, un *string* (cadena de text) és una **seqüència immutable** de caràcters i per tant, no es poden modificar directament un cop creades. Per declarar una string, es pot utilitzar cometes simples (") o dobles ("). També es poden utilitzar cometes triples (""" """) per a cadenes multilínia (normalment s'utilitza aquest format per documentar el nostre codi).

```
In [ ]: # String simple amb cometes simples
str1 = 'Hola'

# String simple amb cometes dobles
str2 = "Mon"

# String multilínia amb cometes triples
str3 = '''Això és una
cadena multilínia.'''
```

En Python no existeix el caràcter (`char`) com a tipus bàsic de dades, ja que un caràcter és simplement un *string* de longitud 1.

```
In [ ]: char1 = 'a'
char2 = "b"
print(type(char1))
print(type(char2))
```

Principals mètodes i funcions de Strings

- `len(string)`: retorna la longitud de la cadena.
- `.upper()`: converteix la cadena a majúscules.
- `.lower()`: converteix la cadena a minúscules.
- `.strip()`: elimina espais en blanc inicials i finals.
- `.replace(old, new)`: substitueix `old` per `new`.
- `.split(sep)`: divideix la cadena en una llista pels separadors `sep`.

```
In [ ]: text = "  Hola, Mon  "
print(len(text))          # 12
print(text.upper())       # '  HOLA, MON  '
print(text.strip())       # 'Hola, Mon'
print(text.replace('o', 'Ø')) # '  HØLa, MØn  '
print(text.split(','))    # ['  Hola', ' Mon  ']
print(text.split(' '))    # ['  Hola', 'Mon  ']
```

En Python es poden arribar a concatenar mètodes (sempre que tingui sentit aplicar un d'arrera l'altre):

```
In [ ]: cad = 'Patata'
cad_resultant = cad.lower().replace('a', '4')
print(cad_resultant)
```

Seria equivalent a fer això:

```
In [ ]: 'Patata'.lower()
```

```
In [ ]: 'patata'.replace('a', '4')
```

Accés als elements d'un String (*slicing*)

Els Strings són seqüències de caràcters, per tant es pot accedir als elements a través del seu índex (començant per 0). També es pot fer *slicing* per obtenir subcadenaes.

```
In [ ]: s = "Python"
print(s[0])    # 'P'
print(s[-1])   # 'n' (últim element)
print(s[1:4])  # 'yth' (elements d'índex 1 a 3)
print(s[:3])   # 'Pyt' (del principi fins a 3, excloent)
print(s[3:])   # 'hon' (de l'índex 3 fins fi)
print(s[::2])  # 'Pto' (del principi al final de 2 en 2)
```

Que un string sigui immutable significa que, un cop creat, el seu contingut no es pot modificar. Això vol dir que no és possible canviar un caràcter concret dins la string mitjançant assignació d'índex, per exemple:

```
In [ ]: cadena = "nuria"
cadena[0] = "N" # Això genera un error TypeError
```

La raó és que les strings són objectes immutables a Python, i qualsevol intent de modificar-ne un caràcter directament dóna error. En canvi, quan es vol canviar part d'una string, cal crear-ne una de nova amb la modificació desitjada, per exemple fent *slicing* i concatenació:

```
In [ ]: nom_cadena = "núria"
nom_cadena = "N" + nom_cadena[1:] # 'Núria'
print(nom_cadena)
```

Així, la variable `nom_cadena` "apunta" ara a una nova string creada a partir de l'anterior `nom_cadena`.

Mostrar missatges per pantalla (funció `print`)

Hi ha diverses maneres de mostrar per pantalla missatges que combinen text i variables en Python,

incloent formes "clàssiques" i avançades:

Concatenació amb operador + (forma clàssica):

```
In [ ]: nom = "Núria"
        edat = 30
        print("Hola " + nom + ", com estàs? Tens " + str(edat) + " anys.")
```

Cal convertir explícitament amb `str()` les variables que no siguin cadenes.

Múltiples arguments de print:

El resultat afegeix espais automàticament entre els arguments i no cal convertir explícitament les variables.

```
In [ ]: print("Hola", nom, ", com estàs? Tens", edat, "anys.")
```

Format amb el mètode .format() (des de Python 2.7 i 3.x):

Permet utilitzar `{}` que es substitueixen pels arguments de `.format()`.

```
In [ ]: print("Hola {}, com estàs? Tens {} anys.".format(nom, edat))
```

També permet indicar el format de representació de números, com ara decimals:

```
In [ ]: import math

        print(math.pi)
        print("El valor de pi és {:.2f}".format(math.pi)) # dos decimal
```

F-strings (disponibles des de Python 3.6):

Sintaxi fàcil i clara que permet indicar variables, expressions i formats.

```
In [ ]: print(f"Hola {nom}, com estàs? Tens {edat} anys.")
```

Donar format als decimals és molt senzill:

```
In [ ]: #Ja hem fet import math anteriorment

        print(f"El valor de pi és {math.pi:.2f}") # 3.14
```

Es poden fer càlculs dins dels `{}` dels f-strings:

```
In [ ]: print(f"{nom}, l'any vinent tindràs {edat + 1} anys.")
```

Aquestes formes permeten concatenar variables i controlar el format dels valors, com el nombre de decimals en floats.

Les f-strings són el mètode més potent i expressiu per al format de cadenes. Estan disponibles a Python des de la versió 3.6 i, a partir de la versió 3.12, incorporen encara més flexibilitat i una millor gestió d'errors.

Aquí un exemple avançat que mostra alguna de les novetats de Python 3.12 (commentaris dins expressions, reutilització de cometes i expressions multilínia):

```
In [ ]: #nom= "Núria" #variable declarada anteriorment
        #edat = 30   #variable declarada anteriorment
```

```

preu_actual = 45.6789

missatge = f"""
Usuari: {nom.upper()} # nom en majúscules
Edat pròxima: {edat + 1} # increment de l'edat

Detalls del producte:
- Preu actual: {preu_actual:.2f} EUR # preu amb 2 decimals
- IVA (21%): {preu_actual * 0.21:.2f} EUR # càlcul de l'IVA

Missatge amb quotes reutilitzades: {f"El nom és \"{nom}\""} # quotes dobles dins el mate
"""

print(missatge)

```

Entrada de dades per teclat

La lectura d'entrada amb la funció `input()` és la forma bàsica d'interactuar amb l'usuari des del teclat a Python.

Quan el programa executa `input()`, s'atura i mostra el missatge (si s'ha posat com argument) esperant que l'usuari introdueixi una resposta pel teclat i premeu la tecla Enter.

El valor retornat per `input()` cal guardar-lo en una variable. Si no ho fem així, el valor introduït "es perd".

```

In [ ]: paraula = input("Introdueix la primera paraula que et vingui al cap: ") # retorna string
print(paraula)

```

Si vols poder treballar amb números (i poder operar amb ells) caldrà transformar-los explícitament.

```

In [ ]: edat_str = input("Introdueix la teva edat: ")
        edat_int = int(edat_str) # convertir a enter

preu_str = input("Introdueix un preu: ")
preu_float = float(preu_str) # convertir a flotant

```

Això permet fer operacions matemàtiques.

```

In [ ]: suma_valors = edat_int + preu_float
print(suma_valors)

```

```

In [ ]: suma_str = edat_str + preu_str
print(suma_str)

```

Arguments en línia de comandes en Python

Quan s'executa un programa Python des del terminal, sovint volem poder passar paràmetres a aquest programa.

Aquests paràmetres s'introdueixen a la línia de comandes després del nom del programa.

Accés als arguments de `sys.argv`

El mòdul `sys` inclou una llista anomenada `argv` que conté tots els arguments passats al programa quan s'executa.

Imagina que des de terminal executem:

```
python executa.py hola 23
```

```
In [ ]: import sys
```

```
In [ ]: sys.argv = ['executa.py', 'hola', '23']
```

```
In [ ]: print("Nombre d'arguments:", len(sys.argv))
        print("Arguments:", sys.argv)
```

Per controlar de manera més sofisticada els arguments, es recomana usar el mòdul `argparse`. Permet definir opcions, descripcions, i ajuda automàtica per als arguments.

Exemple bàsic:

```
#contingut fitxer exemple_argparse.py
import argparse

parser = argparse.ArgumentParser(description="Exemple d'ús d'arguments")
parser.add_argument("-v", "--verbose", help="mostra informació detallada",
                    action="store_true")
args = parser.parse_args()

if args.verbose:
    print("Mode detallat activat")
```

Aquest tutorial oficial de la documentació de `argparse` de Python i explica des de la configuració bàsica fins a exemples més avançats:

<https://docs.python.org/es/3/howto/argparse.html>

Biblioteca estàndard de Python

En Python, una llibreria (o "mòdul") és un conjunt de funcions i recursos que es poden importar per ampliar la funcionalitat bàsica del llenguatge.

La gran fortalesa de Python és el seu ecosistema de llibreries tant de la biblioteca estàndard com de tercers.

Importar aquestes llibreries

Per tal de poder utilitzar aquestes llibreries especialitzades les hem d'importar al nostre projecte Python.

```
import math
```

Per utilitzar una funció d'una llibreria importada, cal posar el nom del mòdul al davant:

```
In [ ]: arrel = math.sqrt(16) # Crida la funció sqrt(arrel quadrada) del mòdul math
        print(arrel)
```

També es pot importar només determinades funcions/elements:

```
In [ ]: from math import sqrt, pi

        print(sqrt(49)) # 7.0
```



```
print(pi)          # 3.141592653589793
```

Una altra tècnica és utilitzar un àlies (per a noms llargs o per convenció). Al llarg del curs anirem aprenent algunes d'aquestes convencions.

```
In [ ]: import random as rnd

print(rnd.randint(1, 5))
```

Mòduls de la llibreria estàndard de Python

La biblioteca estàndard és un conjunt molt ampli de mòduls i paquets que es distribueixen juntament amb la instal·lació oficial de Python.

Aquí en podeu consultar tota la documentació: <https://docs.python.org/3/library/index.html>

Expressions regulars (re)

Permet cercar, substituir i analitzar textos mitjançant patrons.

```
In [ ]: import re

resultat = re.findall(r'\d+', "L'any 2025 té 365 dies i 12 mesos.")
print(resultat) # ['2025', '365', '12']
```

```
In [ ]: text1 = "El meu id de treballador és 555-1234 i el del meu company de despatx és 666-5678"

# Buscar totes les ocurrences de números amb format xxx-xxxx
filtre = re.findall(r'\d{3}-\d{4}', text1)

print(filtre) # ['555-1234', '666-5678']
```

Funcions matemàtiques (math)

Funcions matemàtiques i constants.

```
In [ ]: #import math #ja s'ha importat anteriorment

print(math.sqrt(25))      # 5.0
print(math.pi)            # 3.141592653589793
print(math.sin(math.pi/2))# 1.0
```

Nombres aleatoris (random)

Serveix per generar nombres i seleccionar elements de manera aleatòria.

```
In [ ]: import random

print(random.randint(1, 10))      # Enter aleatori entre 1 i 10
print(random.choice(['cara', 'creu']))# 'cara' o 'creu' a l'atzar
print(random.random())            # Decimal aleatori entre 0 i 1
```

Dates i hores (datetime)

Permet treballar amb dates, hores i intervals de temps.

```
In [ ]: from datetime import datetime, timedelta

avui = datetime.now()
print("Data i hora actual:", avui)
```

```
demà = avui + timedelta(days=1)
print("Demà:", demà.strftime("%d-%m-%Y"))
```

Sistema Operatiu (os)

Funcions per interactuar amb el sistema operatiu: gestió de fitxers , directoris, etc.

```
In [ ]: import os

print("Directori actual:", os.getcwd())
print("Llista de fitxers:", os.listdir('.'))
```

Paràmetres i Execució (sys)

Informació i mètodes relacionats amb el sistema operatiu.

```
In [ ]: #import sys #està important anteriorment

print("Versió de Python:", sys.version)
print("Arguments rebuts:", sys.argv)
```